

Compression

Model Compression

Опр. **Сжатие моделей (model compression)** - уменьшение размера модели (с сохранением поведения / минимизацией потери качества).

Цели:

- экономия ресурсов для хранения модели;
- экономия вычислительных ресурсов для инференса (RAM, VRAM, energy);
- ускорение инференса/обучения (inference time, latency, throughput)
- сохранение поведения модели / минимизация потери качества.

Подходы к сжатию

Основные подходы:

- knowledge distillation
- pruning
- quantization

Дополнительные (не будем рассматривать):

- *low-rank factorization*
- *NAS*

Метрики

- Метрики вычислительной эффективности
 - FLOPs (операции с плавающей запятой)
 - MACs (операции умножения-накопления) [1 MAC $\approx$ 2 FLOPs (*, +)]
- Скорость инференса
 - Задержка (latency)
 - Пропускная способность (throughput)
- Потребление памяти
 - Размер модели (количество параметров)
 - Память для промежуточных активаций
- Потребление энергии
- Метрики оценки качества модели
- Робастность

Постановка задачи

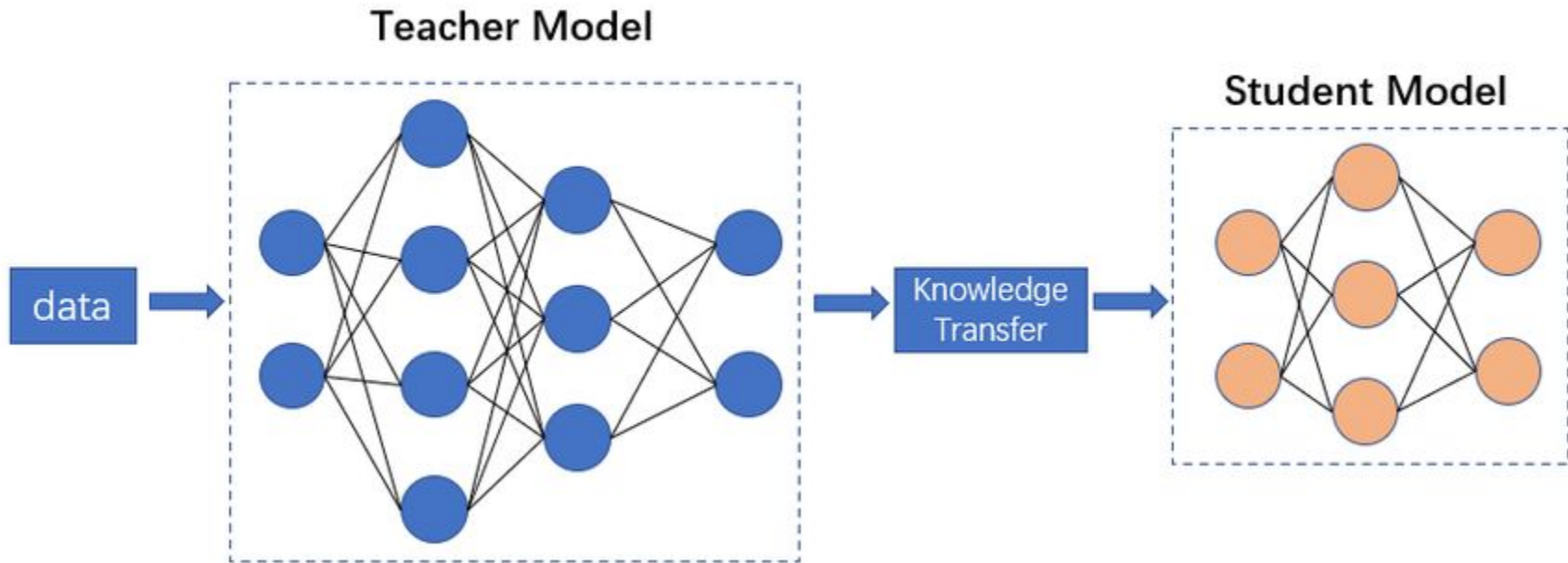
Постановки задачи построения сжатой модели:

- дана большая модель, надо ее сжать
- строим одновременно замечательную хорошую модель и ее уменьшенные копии
- строим маленькую модель, используем построение большой модели как вспомогательное средство

Knowledge Distillation

Определение

Опр. Knowledge distillation - техника переноса знаний от большей модели (учителя) к меньшей (ученику).

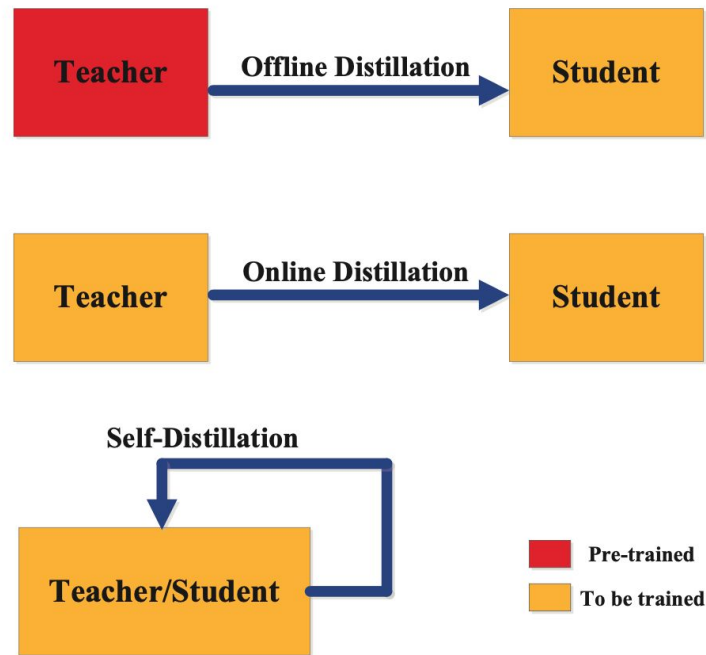


Схемы обучения

Схемы обучения:

- **offline distillation**
обучаем ученика по обученному учителю
- **online distillation**
обучаем ученика и учителя одновременно
- **self-distillation**
обучаем одну модель в качестве учителя и ученика

Note: теор. обоснование само-дистилляции [Mohabi et al., 2020](https://arxiv.org/abs/2006.05525)



source: <https://arxiv.org/abs/2006.05525>

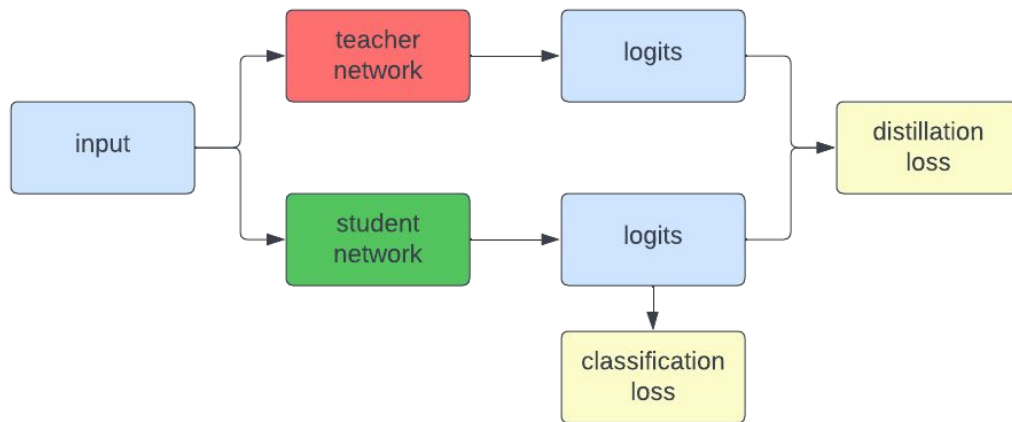
Как переносить знания?

Для переноса знания можно согласовывать:

- выходы (логиты)
- веса
- латентные представления
- отношения

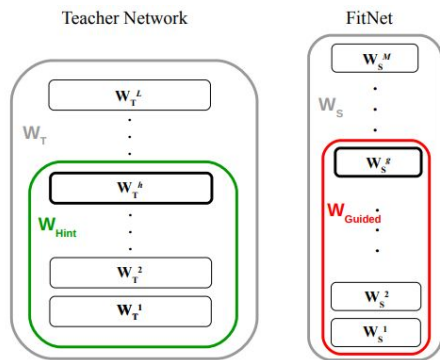
Согласование логитов

Distillation loss: CE, MSE

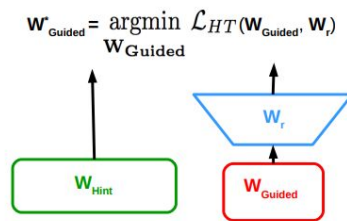


Согласование весов

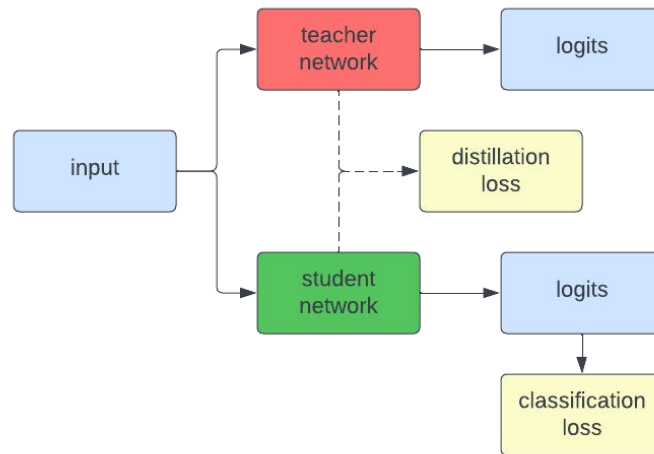
Distillation loss: MSE



(a) Teacher and Student Networks



(b) Hints Training

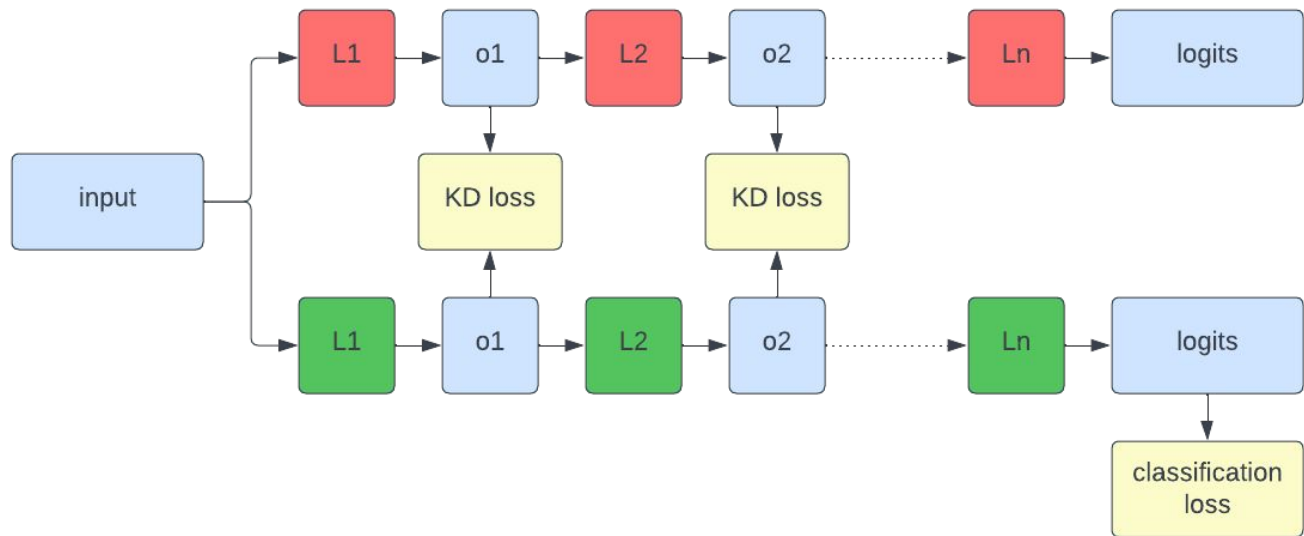


Согласование латентных представлений

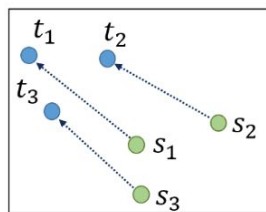
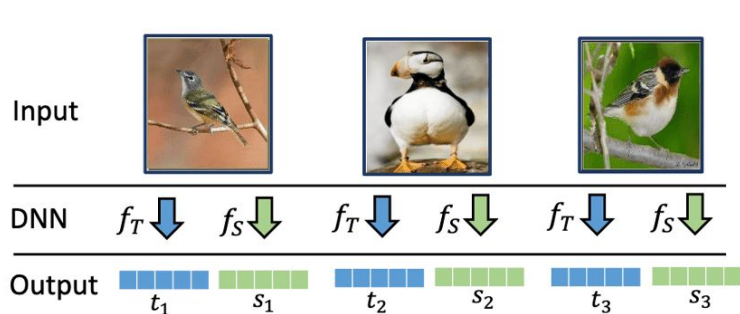
Distillation loss: MSE, CE, MMD

Note: SimKD (CVPR 2022) - был SOTA методом, несмотря на простоту.

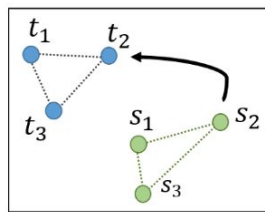
<https://github.com/DefangChen/SimKD>



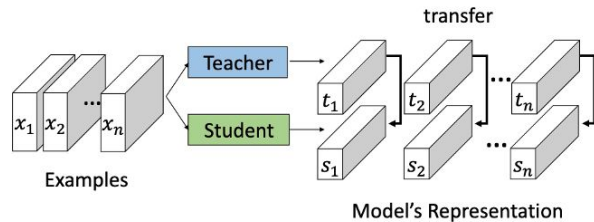
Согласование отношений



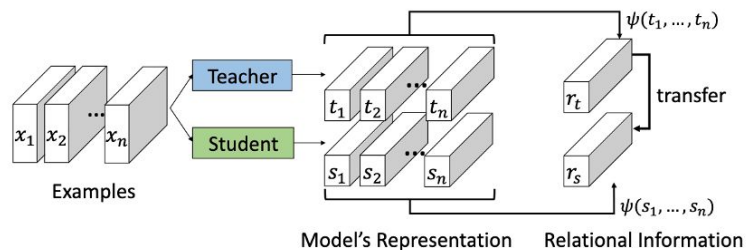
Point to Point
Conventional KD



Structure to Structure
Relational KD



Individual Knowledge Distillation

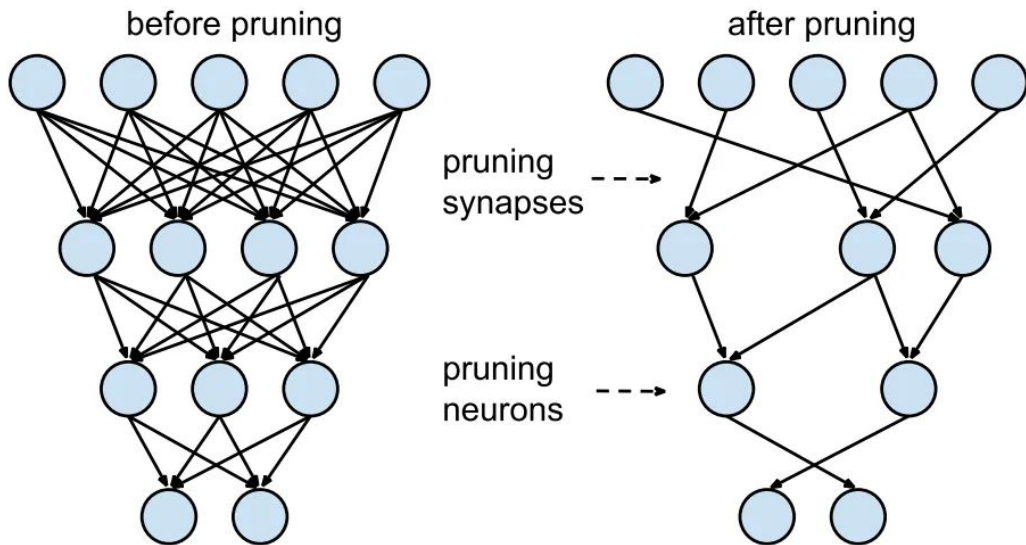


Relational Knowledge Distillation

Pruning

Pruning

Опр. **Прунинг (pruning)** - удаление нейронов / синапсов.



Гиперпараметры прунинга

- гранулярность
- коэффициент прореживания (pruning ratio)
- критерий
- тьюнинг

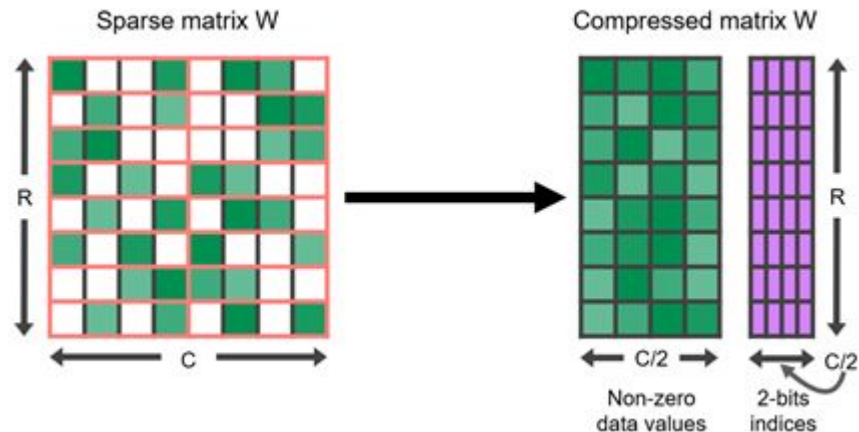
Гранулярность

- неструктурированный (unstructured = fine-grained) прунинг
 - удаление произвольных параметров
- структурированный (structured = coarse-grained) прунинг
 - удаление структур параметров, соответствующих накладываемым ограничениям
 - N:M pruning
 - group pruning
 - vector pruning
 - filter pruning
 - channel pruning

N:M pruning

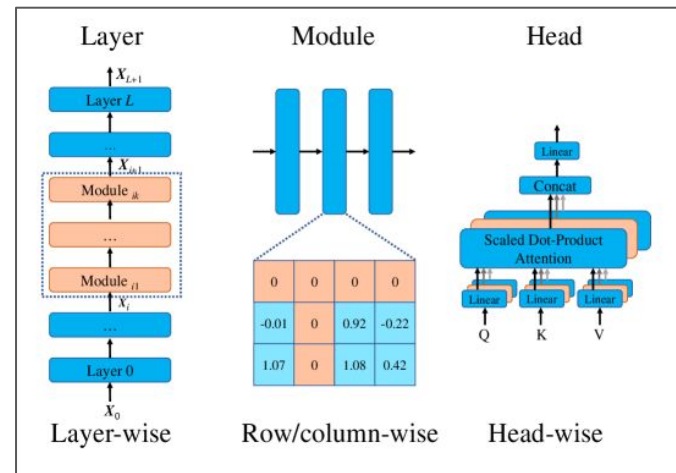
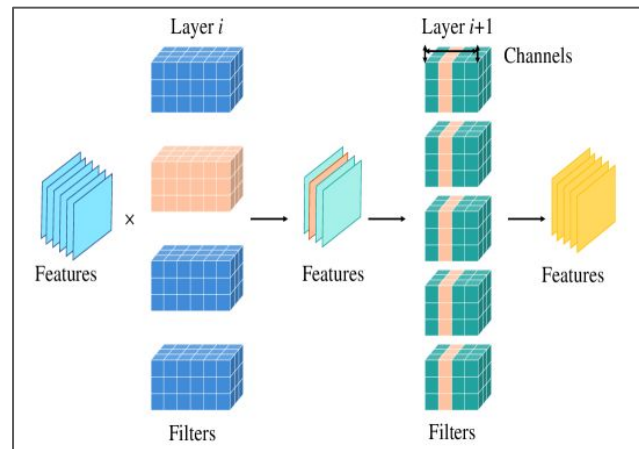
Опр. при N:M прунинге в каждой группе из M весов оставляем N штук.

Note: 2:4 поддерживается Nvidia Ampere.



В зависимости от архитектуры

- CNNs
 - filter pruning
 - channel pruning
- Transformers
 - attention-head pruning
- Skip-connections
 - layer pruning



Сравнение

Unstructured

- + бОльшая степень сжатия с меньшей потерей качества предсказания
- - сложнее обеспечить оптимизацию с т. зр. железа

Structured

- + меньшая степень сжатия с бОльшей потерей качества предсказания
- - проще обеспечить оптимизацию с т.зр. железа

Pruning ratio

Коэффициент прореживания (pruning ratio) r задает % удаляемых весов.

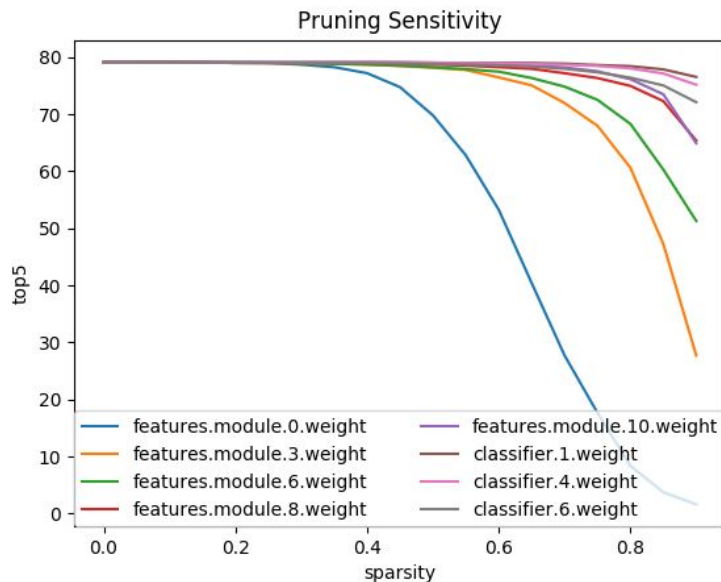
- равномерный прунинг: r_L каждого слоя L совпадает с r модели
- неравномерный прунинг: r_L каждого слоя L выбирается индивидуально, но так чтобы в совокупности по модели выходило r

Как делать неравномерный прунинг?

Неравномерный прунинг

Жадный подход:

- замерим чувствительность каждого слоя ($\text{quality}(r_L)$)
- выберем наибольшие r_L , не превышающие пороговое проседание качества



source: <https://intellabs.github.io/distiller/pruning.html>

Критерии прунинга

Критерий прунинга определяет как выбирать, какие именно веса удалять.

Рассмотрим следующие критерии:

- magnitude-based
 - L_p -norm
- saliency
 - Taylor pruning
- learning to prune
 - scaling factors

Больше существующих критериев:

<https://arxiv.org/abs/2308.06767>

<https://arxiv.org/abs/2303.00566>

L_p -norm

Удаляется структура с наименьшей L_p нормой.

$$\|\mathbf{w}\|_p = \left(\sum_{i=1}^n |w_i|^p \right)^{1/p}$$

оригинал

4	4
1	-5

L_1

4	4	8
1	-5	6

$(L_2)^2$

4	4	32
1	-5	37

Taylor pruning

Идея: Пруним веса, удаление которых мало меняет лосс.

Для подсчета влияния веса на лосс воспользуемся разложением функции в ряд Тейлора:

$$\mathcal{L}(w + \Delta w) \approx \mathcal{L}(w) + \frac{\partial \mathcal{L}}{\partial w} \Delta w + \mathcal{O}(\Delta w^2)$$

Возьмем:

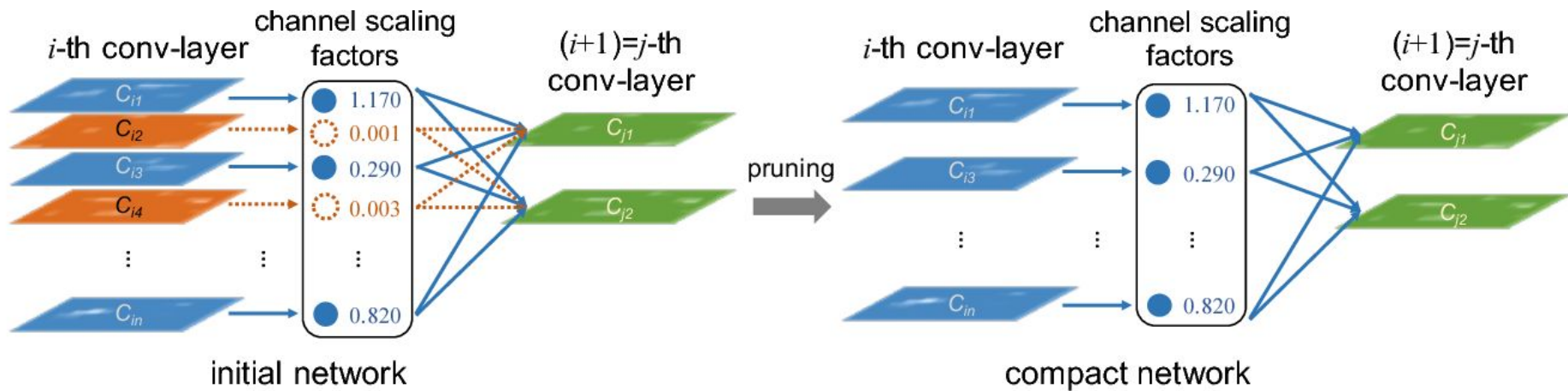
$$\Delta w = -w$$

В предположении о малой магнитуде весов, получаем формулу изменения лосса:

$$\Delta \mathcal{L} \approx -w \cdot \frac{\partial \mathcal{L}}{\partial w}$$

Learning to Prune

- добавляем факторы масштаба
- обучаем с ними
- пруним структуры с маленькими факторами масштаба



pruning vs training

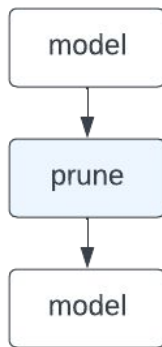
Когда можно прунить относительно обучения:

- PBT (pruning before training): пруним -> обучаем
- PDT (pruning during training): пруним & обучаем
- PAT (pruning after training): обучаем -> пруним

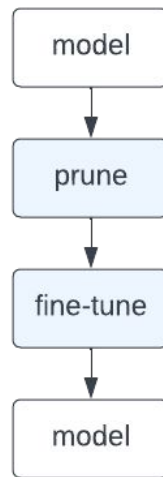
pruning vs fine-tuning

- one-shot
 - PTP (post-training pruning)
 - PAT
- iterative pruning

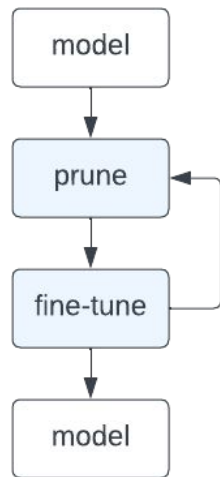
PTP=one-shot



classic PAT



iterative pruning



Quantization

Quantization

Опр. **Квантизация (quantization)** - понижение численной точности (reduction of numerical precision) параметров/активаций.

- + меньше памяти
- + более дешевые (=быстрые) операции
- ошибка квантизации (quantization error) при конвертации
- менее точные вычисления

Числовые типы данных

Типы данных:

- целочисленные (integers)
- с фиксированной точкой (fixed point)
- с плавающей точкой (floating point)

В квантизации используют:

- FP32
- FP16, BF16
- INT8
- INT4



Integers

- unsigned n-bit integer

- $a = a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0$
- диапазон: $0 \dots 2^n - 1$



- signed n-bit integer

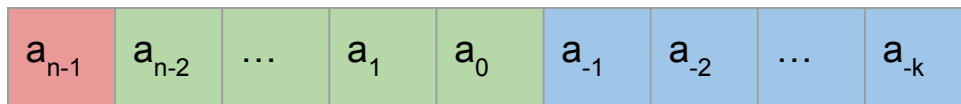
- $a = -a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0$
- диапазон: $-2^{n-1} \dots 2^{n-1} - 1$



Fixed-point

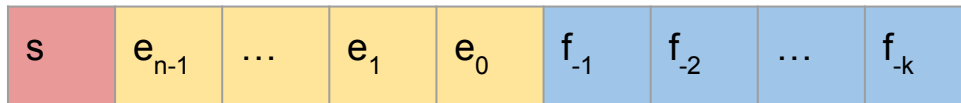
$$a = -a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0 + a_{-1}2^{-1} + a_{-2}2^{-2} + \dots + a_{-k}2^{-k}$$

- знак: 1 bit
- целая часть: n-bit
- дробная часть: k-bit



Floating-point

- знак (sign) s : 1 bit
- порядок (exponent) E : n -bit $\rightarrow$ range
- мантисса (fraction) F : k -bit $\rightarrow$ precision



(Sub)Normal Numbers

normal numbers ($E \neq 0$)

$$a = (-1)^s \times (1 + F) \times 2^{E-\text{bias}}$$

$$F = f_{-1}2^{-1} + f_{-2}2^{-2} + \dots + f_{-k}2^{-k}$$

$$E = e_{n-1}2^{n-1} + e_{n-2}2^{n-2} + \dots + e_02^0$$

subnormal numbers ($E = 0$)

$$a = (-1)^s \times F \times 2^{1-\text{bias}}$$

$$F = f_{-1}2^{-1} + f_{-2}2^{-2} + \dots + f_{-k}2^{-k}$$

Floating-point types

CUDA GPU поддерживают что-то из:

	name	spec	s (bits)	E (bits)	F (bits)	max
FP64	Double Precision	IEEE-754	1	11	52	$\sim 10^{30}_8$
FP32	Single Precision	IEEE-754	1	8	23	$\sim 10^{38}$
FP16	Half Precision	IEEE-754	1	5	10	$\sim 10^4$
BF16	Brain Float	Google	1	8	7	$\sim 10^{38}$

Downcasting effects

меньше битов ->

- меньше памяти
- дешевле операции
- меньше диапазон и/или точность

Отсюда эффекты квантизации.

Симметричная линейная квантизация

Основное свойство: $0 \rightarrow 0$.

Формула: $R = sQ$, где

R - исходный тензор (fp32)

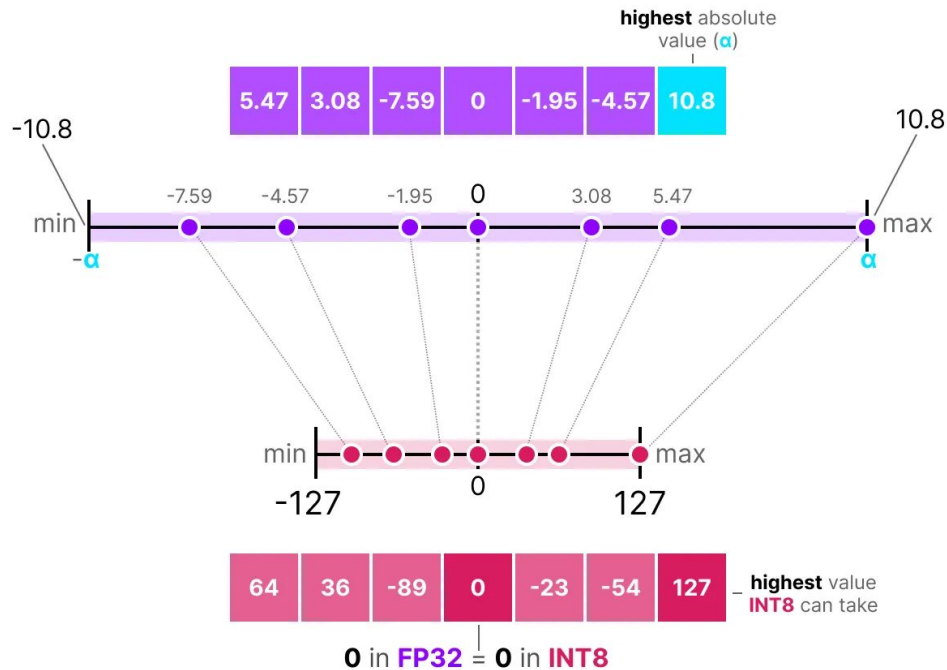
Q - квантованный тензор (int8)

s - scaling factor (fp32)

Обратное преобразование:

$Q = \text{round}(R / s)$

$S = \max(\text{abs}(R)) / \max(\text{abs}(Q))$



Асимметричная линейная квантизация

Основное свойство: $0 \rightarrow z$.

Формула: $R = s(Q - z)$, где

R - исходный тензор (fp32)

Q - квантованный тензор (int8)

s - scaling factor (fp32)

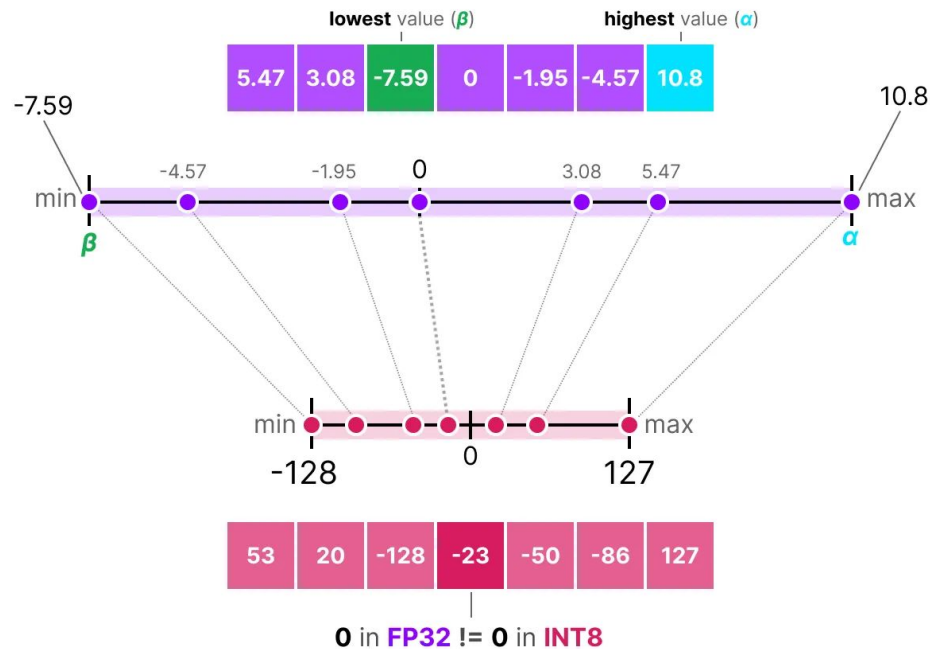
z - zero point (int8)

Обратное преобразование:

$Q = \text{round}(R / s + z)$

$S = (\max(R) - \min(R)) / (\max(Q) - \min(Q))$

$z = \text{round}(\min(Q) - \min(R) / s)$



Матричное умножение

$$Y = WX = s_w(Q_w - z_w)s_x(Q_x - z_x) = s_ws_x(Q_wQ_x - z_wQ_x - z_xQ_w + z_wz_x)$$

Матричное умножение FP32 -> матричное умножение INT8

Повышение качества

- Посттренировочная квантизация (PTQ, Post-Training Quantization) - Модель обучается в FP32, затем сжимается.
 - Можно потренировать сжатую модель.
- Квантизация во время обучения (QAT, Quantization Aware Training) - Модель обучается с имитацией квантования (fake quantization).
- Mixed-Precision Quantization - Разная квантизация на разных слоях.

Заключение

Сравнение

	compression ratio	speed up	hardware support	ease of use	training overhead	accuracy drop	notes
str. pruning	moderate/high	moderate	moderate	moderate	moderate	moderate	easily recoverable with fine-tuning
distillation	moderate	moderate	very high	very easy	high	low	with compatible teacher/student sizes
quantization	high	high	high	easy	moderate	moderate	might not be recoverable with fine-tuning

Sources

- J. Gou et al., 2021. Knowledge Distillation: A Survey <https://arxiv.org/pdf/2006.05525>
- H. Cheng et al., 2024. A Survey on Deep Neural Network Pruning-Taxonomy, Comparison, Analysis, and Recommendations <https://arxiv.org/abs/2308.06767>
- Y. He, L. Xiao, 2023. Structured Pruning for Deep Convolutional Neural Networks: A survey <https://arxiv.org/abs/2303.00566>
- <https://efficientml.ai>
- <https://learn.deeplearning.ai/courses/quantization-fundamentals>
- <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

Спасибо за внимание!